

COOL Grammar: Production Rules, Precedence & Associativity

COOL Compiler — frontend/parser — LL(1) recursive descent grammar

Scope of this document

This document fixes the exact production rules the hand-written recursive descent parser implements, and the precedence/associativity rules that resolve what would otherwise be ambiguity in the expression grammar. Every rule below is written so that it maps directly to one parsing function — each `*/+/?` in a rule corresponds to one loop or one conditional in the corresponding C function, with no intermediate nonterminal invented purely to satisfy a parsing algorithm.

Notation

Symbol	Meaning
UPPERCASE	A terminal (token type produced by the lexer), e.g. CLASS, ID, INT_CONST.
lowercase	A nonterminal, defined by its own rule elsewhere in this document.
A B	A followed by B, in sequence.
A B	Either A or B.
[X]	X appears zero or one time (optional).
X*	X appears zero or more times.
X+	X appears one or more times.
'x'	A literal token spelled x, e.g. '+' for TOKEN_PLUS, '<-' for TOKEN_ASSIGN.

Top-level structure

```
program      ::= class_decl+

class_decl   ::= CLASS TYPE [ INHERITS TYPE ] '{' feature* '}' ';'

feature      ::= ID '(' formals ')' ':' TYPE '{' expr '}' ';'  (* method *)
               | ID ':' TYPE [ '<-' expr ] ';'                (* attribute *)

formals      ::= [ formal ( ',' formal ) * ]
formal       ::= ID ':' TYPE
```

A program is simply one or more class declarations — no left recursion here, so this maps directly to a loop in `parse_program` that keeps calling `parse_class` until the token list is exhausted. Feature, formal, and argument lists are all written with explicit `*/?` rather than left recursion, so each becomes one straightforward loop, not a rewritten grammar fragment.

Expression grammar, by precedence tier (loosest to tightest)

Each tier below is its own nonterminal and its own parsing function. A tier only descends into the next-tighter tier for its operands, which is exactly what encodes precedence without needing a precedence table at parse time.

```

expr          ::= assign_expr

assign_expr   ::= ID '<-' assign_expr
               |   not_expr

               (* right-associative; one extra token of lookahead after ID
               decides this alternative vs. falling through to not_expr *)

not_expr      ::= NOT not_expr
               |   compare_expr

compare_expr  ::= add_expr ( ( '<' | '<=' | '=' ) add_expr )?

               (* non-associative: at most ONE comparison operator allowed;
               'a < b < c' is not a legal COOL expression *)

add_expr      ::= mul_expr ( ( '+' | '-' ) mul_expr )*

mul_expr      ::= isvoid_expr ( ( '*' | '/' ) isvoid_expr )*

isvoid_expr   ::= ISVOID isvoid_expr
               |   neg_expr

neg_expr      ::= '~' neg_expr
               |   dispatch_expr

dispatch_expr ::= primary ( '.' ID '(' args ')'
                           | '@' TYPE '.' ID '(' args ')' )*
```

The `add_expr` and `mul_expr` rules are the left-recursion rewrite discussed in the tradeoffs document: COOL's own specification states these left-recursively (`expr ::= expr '+' expr`), which a top-down parser cannot consume directly. Restating them as `tighter_tier ( op tighter_tier )*` removes the left recursion and maps to one while loop per tier, folding left to right as it runs — which is exactly what produces a correctly left-associative tree.

Primary expressions

```

primary       ::= INT_CONST
               |   STR_CONST
               |   BOOL_CONST
               |   ID '(' args ')'           (* self-dispatch *)
               |   ID                       (* object reference *)
```

```

| '(' expr ')'
| '{' expr ( ';' expr )* ';' '}'
| IF expr THEN expr ELSE expr FI
| WHILE expr LOOP expr POOL
| LET let_binding ( ',' let_binding )* IN expr
| CASE expr OF case_branch+ ESAC
| NEW TYPE

args          ::= [ expr ( ',' expr )* ]

let_binding   ::= ID ':' TYPE [ '<-' expr ]

case_branch   ::= ID ':' TYPE '=>' expr ';'

```

Both ID '(' and a bare ID start with the same token, which is the second of the two spots requiring one extra token of lookahead rather than a formal grammar transformation (see the tradeoffs document). Every compound construct here is explicitly closed by its own terminal keyword (FI, POOL, ESAC, the closing '}'), which is precisely what keeps this grammar free of the dangling-construct ambiguity that would otherwise force a lookahead-heavier or bottom-up approach.

Precedence table

Tier (loosest → tightest)	Operator(s)	Associativity
1	'<-' (assignment)	Right
2	NOT	Right (unary)
3	'<' '<=' '='	Non-associative — at most one per expression
4	'+' '-'	Left
5	'*' '/'	Left
6	ISVOID	Right (unary)
7	'~'	Right (unary)
8	'@' (static dispatch)	Left
9	'.' (dispatch)	Left — tightest

Worth double-checking at implementation time: NOT sits *above* assignment but *below* the comparison operators. This means not `a < b` parses as not `(a < b)`, not `(not a) < b` — the semantically sensible reading (negating a whole comparison), and only correct if the grammar places NOT exactly at this tier rather than binding it tighter, as intuition might otherwise suggest.

What this grammar deliberately leaves unchecked

Consistent with the phase boundary already established for the lexer, this grammar encodes shape only. None of the following are grammar-level concerns, and none are checked by the parser — all are deferred to semantic analysis:

- Whether a referenced TYPE name has actually been declared anywhere in the program.
- Whether an ID used as an object reference has been declared in the current scope.
- Whether the operand types on either side of an operator (e.g. '+' or a comparison) are actually compatible.
- Whether a method call supplies the correct number or types of arguments.
- Whether a class's declared parent in an INHERITS clause actually exists, or whether inheriting from it would create a cycle.